The Distributed Workflow Library

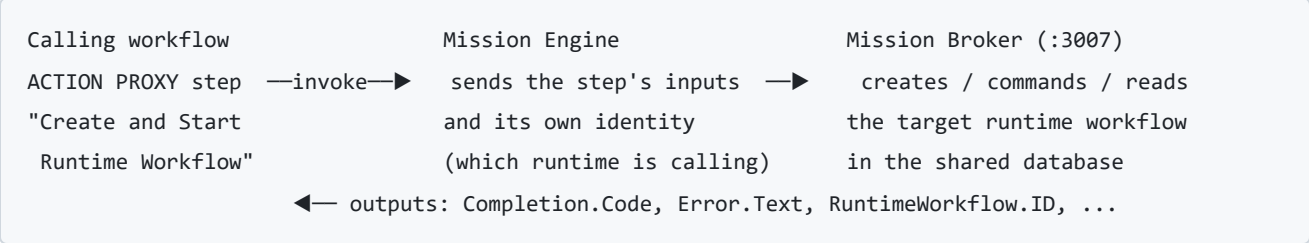
The **Distributed Workflow Action Library** lets a running workflow create, start and control other runtime workflows, in its own work center or in another one. It follows the concepts proposed by the ISA 113 committee on Distributed Workflow Management.

This guide covers what the library contains, how to set it up, how to use each action in a workflow, and two sample SCRIPT steps that convert between single values and the Value Property list form that the library reads and writes.

- 1. How it works
- 2. Setting it up
- 3. The actions
- 4. Inputs and outputs in detail
- 5. Building a workflow that uses the library
- 6. Sample SCRIPT steps
- 7. Limits and troubleshooting

---

1. How it works



- The actions live in the GLOBAL Mission Designer library **Distributed Workflow Action Library**. Its environment library, **Distributed Workflow Environment Library**, holds the environment **Distributed Workflow Management**, which includes all ten actions.
- At runtime the actions are carried out by **Mission Broker**, a small service in the Mission Control stack (`http://mission-broker:3007`). It speaks the same action-server protocol as Mission Autopilot, so a work center connects to it exactly as it connects to any other action server.
- Because the Mission Engine tells the action server which runtime is calling, Mission Broker knows the caller's work center (so a blank work center means "my own"), records which runtime created which ("Created by RW-12" in Mission Commander), and checks that the caller is allowed to act.
- Every action except **Wait for Runtime Workflow** completes in well under a second.
- A refusal is never a step failure. The action completes normally with a non-zero `Completion.Code` and an explanation in `Error.Text`, so the workflow can branch on it.

---

2. Setting it up

2.1 In Mission Commander (administrator, once per work center)

1. **Allow the server.** Open **Settings** and add `http://mission-broker:3007` to the action-server allowlist (required when Action server trust is Strict).
2. **Load the environment.** Load the *Distributed Workflow Management* environment package into each work center whose workflows will call the actions (**Load packages...**). The package is in this repository: `MissionBroker/designer-library/Distributed Workflow Management(1.0.0).WFenvirX`. You can also export it from Mission Designer (the environment's **Export**).
3. **Bind it.** On the work center's **Env Specs** tab, set the *Distributed Workflow Management* environment's **Action Server** to `http://mission-broker:3007` and click **Save**.
4. **Grant cross-work-center control (only if needed).** A workflow may always control runtimes in its own work center. To reach another work center, open the **Control** tab of the work center whose runtimes will do the controlling and add a grant for the target work center:
    - **Only runtimes it created** — it may create runtimes in the target work center and command the ones it (or a runtime it created) started.
    - **Any runtime** — it may also command runtimes it did not create.

    Reading a runtime's state or outputs, and waiting for it, needs either kind of grant.

The Master Workflow that will be created must also be loaded into the target work center.

2.2 In Mission Designer (workflow author)

1. Open the workflow and link the **Distributed Workflow Management** environment to it (from the GLOBAL **Distributed Workflow Environment Library**).
2. Drag the environment's button, or a generic **Env Action** step, onto the canvas and pick an action in the **Action Reference** dropdown. The step's inputs and outputs fill in from the action.
3. Bind each input and output as described in section 4.

If a Mission Designer doesn't have the libraries yet, import `MissionBroker/designer-library/Distributed Workflow Environment Library(1.0.0).WFenvirLibX`; it brings in the action library too. The folder's README lists every package and where it goes.

---

3. The actions

Action	Inputs	Outputs (besides Completion and Error)	What it does
Create Runtime Workflow	`WorkCenter.Name`, `Workflow.Name`, `StartingParameters.Value`	`RuntimeWorkflow.ID`, `RuntimeWorkflow.Label`	Creates a runtime workflow and leaves it unstarted (state `Created`)
Create and Start Runtime Workflow	same	same	Creates a runtime workflow and starts it
Start Runtime Workflow	`RuntimeWorkflow.ID`	—	Starts a created runtime
Stop Runtime Workflow	`RuntimeWorkflow.ID`	—	Stops a runtime (orderly)
Abort Runtime Workflow	`RuntimeWorkflow.ID`	—	Aborts a runtime (immediate)
Suspend Runtime Workflow	`RuntimeWorkflow.ID`	—	Pauses a running runtime
Unsuspend Runtime Workflow	`RuntimeWorkflow.ID`	—	Resumes a suspended runtime
Get Runtime Workflow Status	`RuntimeWorkflow.ID`	`Runtime.State`	Reads the runtime's current state
Wait for Runtime Workflow	`RuntimeWorkflow.ID`	`Runtime.State`	Waits until the runtime finishes, then returns its final state
Get Runtime Workflow Outputs	`RuntimeWorkflow.ID`	`Runtime.State`, `Outputs.Value`	Reads the runtime's workflow output parameters as a Value Property list

Every action also returns:

`Completion.Code`	Meaning
0	No error
1	Other error
2	Not found (work center, workflow or runtime)
3	Not permitted (no control grant, or the runtime was not created by the caller's lineage)

`Completion.Code`	**Meaning**
4	Wrong state (for example Suspend on a runtime that is not running)
5	Invalid input (bad starting parameters, an ambiguous work-center name)

`Error.Text` is blank on success and always explains a non-zero code.

3.1 What each command does in each state

`Runtime.State` uses the library's own words: **Created, Starting, Running, Suspending, Suspended, Unsuspending, Stopping, Aborting, Completed, Stopped, Aborted, Errored, Unknown**.

State	**Start**	**Stop**	**Abort**	**Suspend**	**Unsuspend**
Created	starts it, 0	**deletes it**, 0	**deletes it**, 0	4	4
Starting	0	aborts it, 0	aborts it, 0	4	4
Running	0 (already)	stops it, 0	aborts it, 0	suspends it, 0	0 (already)
Suspending / Suspended	0	stops it, 0	aborts it, 0	0 (already)	resumes it, 0
Unsuspending	0	4 (retry)	aborts it, 0	4	0
Stopping	4	0 (already)	aborts it, 0	4	4
Aborting	4	4	0 (already)	4	4
Completed / Stopped / Aborted / Errored	4	0 (already finished)	0 (already finished)	4	4
Unknown (not found)	2	2	2	2	2

A runtime that was created but never started has no history, so stopping or aborting it removes it entirely.

3.2 Wait for Runtime Workflow

- Completes with code 0 and the target's **final** state (Completed, Stopped, Aborted or Errored) when the target finishes. Branch on `Runtime.State` to tell a good finish from a bad one.
- Completes at once if the target has already finished; with code 2 if it no longer exists.
- If the waiting workflow is itself stopped, the wait ends with code 4. Aborting the waiting workflow ends the wait with it.
- It has **no time limit**: it waits for as long as the target runs. (Other actions give up after 30 seconds, which they never need.)

3.3 Get Runtime Workflow Outputs

Returns the target's **workflow output parameters** — the values it has now if it is still running, or the values it finished with. `Outputs.Value` is a Value Property list (see section 4.1). A runtime that finished before this feature was installed has no recorded outputs and returns code 2.

---

4. Inputs and outputs in detail

4.1 The Value Property list form

When a step input is bound to a **whole** Value Property (its name, not one of its entries), the Mission Engine passes the property as a JSON array of name/value pairs. The `Description` entry is always left out. For a property `ChildStart` with entries `Batch`, `Qty` and `Description`:

```
[
  { "name": "Batch", "value": "B-17" },
  { "name": "Qty", "value": "40" }
]
```

All values are strings. This is the format the library reads for starting parameters and returns as a runtime's outputs. It is also the format referred to as the "Value Type form".

4.2 `WorkCenter.Name`

Value	Means
(blank)	The calling workflow's own work center
`Line 1`	The work center named `Line 1`; the name must be unique in the site
`Plant/Area A/Line 1`	A path; always unambiguous. Use it when names repeat (a repeated bare name returns code 5)

4.3 `Workflow.Name`

The Master Workflow's name, loaded in the target work center. When several versions are loaded, the library uses the **in-force Effective** version (effective from a date that has passed); if there is none, the newest **Approved**, then **In Review**, **In Test**, then **Draft**. Superseded and Obsolete versions are never used.

4.4 `StartingParameters.Value`

The created workflow's starting parameters, in any of three forms:

Form	Example
Blank	No starting parameters (the workflow's defaults apply)
`name=value` pairs, separated by `;`	`Batch=B-17; Qty=40`
JSON object	`{"Batch":"B-17","Qty":"40"}`
Value Property list	`[{"name":"Batch","value":"B-17"},{"name":"Qty","value":"40"}]`

The list form arrives when you bind the input to a whole Value Property whose entry names are the parameter names, or when you bind it to an entry that holds such a list (as the Combine script produces).

Every name must be one of the created workflow's starting parameters; an unknown name returns code 5.

> **One quirk.** A whole Value Property with a single entry named `Value` is read as the `StartingParameters` property itself: the text of its `Value` entry is used as the parameters. That is what lets you bind either `ChildStart` or `ChildStart.Value` when `ChildStart.Value` holds the list. So don't name a starting parameter `Value`.

4.5 `RuntimeWorkflow.ID`

The create actions return two values:

- `RuntimeWorkflow.ID` — the runtime's unique ID (a UUID). It stays valid after the runtime finishes and moves to history.
- `RuntimeWorkflow.Label` — the readable Runtime ID shown in Mission Commander and Pilot, such as `RW-12`.

The command, status, wait and outputs actions accept the ID, a label (`RW-12`, in the caller's work center) or a work-center-qualified label (`Line 1/RW-12`, or a full path).

The simplest pattern is to bind the create action's `RuntimeWorkflow` output to a whole Value Property (for example `Child`). Its entries `Child.ID` and `Child.Label` are then filled in, and later steps pass `Child.ID` to their `RuntimeWorkflow.ID` input.

4.6 Binding outputs

- Binding an output to an **entry** (`Result.Code`) stores the raw value there. For `Outputs`, that is the whole list as one JSON string.
- Binding an output to a **whole Value Property** stores the list **and** writes each pair as an entry of that property. For example, `Outputs` bound to `ChildResults` writes `ChildResults.Yield`, `ChildResults.Lot` and so on, one entry per output parameter of the child workflow.

---

5. Building a workflow that uses the library

A typical "run a job elsewhere and use its result" workflow:

```
START
  │
  ▼
SCRIPT  "Combine starting parameters"      (section 6.1)  ──▶ ChildStart.Value
  │
  ▼
ACTION PROXY  Create and Start Runtime Workflow
    WorkCenter.Name           = "Packaging/Line 4"        (text)
    Workflow.Name             = "Fill Drums"               (text)
    StartingParameters.Value  ← ChildStart.Value           (source)
    outputs: Completion → Create, Error → Create, RuntimeWorkflow → Child
  │
  ▼
SELECT 1  on Create.Code  ── "0" ──▶ continue      ── otherwise ──▶ report Create.Text
  │
  ▼
ACTION PROXY  Wait for Runtime Workflow
    RuntimeWorkflow.ID        ← Child.ID
    outputs: Runtime → Final
  │
  ▼
SELECT 1  on Final.State  ── "Completed" ──▶ continue  ── otherwise ──▶ handle failure
  │
  ▼
ACTION PROXY  Get Runtime Workflow Outputs
    RuntimeWorkflow.ID        ← Child.ID
    outputs: Outputs → ChildResults.Value
  │
  ▼
SCRIPT  "Split the child's outputs"        (section 6.2)  ──▶ single values
  │
  ▼
END
```

Tips:

- **Check `Completion.Code` after every action.** A SELECT 1 step on the code (or a step input bound to it) is the usual way. Show `Error.Text` to the operator, or record it, when it is not `0`.

- **Create, then Start**, when something must happen between the two (for example recording the new runtime's label before it begins). Use **Create and Start** otherwise.
- **Fan out** by placing several create actions in parallel branches, then a Wait per child.
- **Suspend / Unsuspend / Stop / Abort** need only `RuntimeWorkflow.ID` and return only Completion and Error.
- **Get Runtime Workflow Status** is a snapshot. To react when a runtime finishes, use **Wait for Runtime Workflow** rather than polling.
- Mission Commander shows the link both ways: the created runtime says **Created by** with the creator's work center and label.

---

6. Sample SCRIPT steps

A SCRIPT step runs a short JavaScript program on the Mission Engine (at most 5 seconds per run). Each of its **Input Parameters** becomes a local variable named by the parameter's name; every value is a string. The script writes results as `output.<output parameter name> = value`, and each output is stored at the target the step's **Output Parameters** name.

6.1 Combine single values into a Value Property list

Use this to build `StartingParameters.Value` for a create action from values the workflow already has, without typing JSON by hand.

Input Parameters — one per starting parameter of the workflow to be created, each **named exactly like that starting parameter** and bound to wherever the value lives (a Value Property entry, a workflow input, a previous step's output, or fixed text). For example:

Name	Source
`Batch`	`Order.Batch`
`Qty`	`Order.Quantity`
`Recipe`	text `R-200`

Output Parameters

Name	Target
`parameters`	`ChildStart.Value`

Script

```
// Combine single values into the Value Property list form:
//   [{"name":"Batch","value":"B-17"}, {"name":"Qty","value":"40"}, ...]
// List every input here. Each name becomes the entry name, so name the step's
// Input Parameters exactly like the starting parameters of the workflow to create.
const values = { Batch, Qty, Recipe }

// true = leave out blank values, so the created workflow uses its own defaults for them.
const skipBlanks = true

const list = []
for (const name of Object.keys(values)) {
  const value = values[name] == null ? '' : String(values[name])
  if (skipBlanks && value.trim() === '') continue
  list.push({ name: name, value: value })
}

output.parameters = JSON.stringify(list)
```

With `Batch = "B-17"`, `Qty = "40"` and `Recipe = "R-200"`, `ChildStart.Value` becomes:

```
[{"name":"Batch","value":"B-17"},{"name":"Qty","value":"40"},{"name":"Recipe","value":"R-200"}]
```

Then bind the create action's `StartingParameters.Value` input to `ChildStart.Value` (or to the whole `ChildStart` property; see the quirk in section 4.4).

To adapt it, add or remove names in the `values` line and add or remove Input Parameters to match. If an entry must have a different name from its variable (for example a name with a space, which is not a valid variable name), write that pair out: `const values = { 'Lot Number': lot, Qty }`.

6.2 Split a Value Property list into single values

Use this to unpack `Outputs.Value` from **Get Runtime Workflow Outputs** (or any other Value Property list) into separate output parameters, each stored where the rest of the workflow needs it.

> If you only need the values as entries of one property, you may not need a script: bind the action's `Outputs` output to a whole Value Property and each pair becomes an entry (section 4.6). Use this script when the list sits in a single entry, when the values must go to different places, or when you want defaults for missing values.

Input Parameters

Name	Source
`list`	`ChildResults.Value` (the entry the list was stored in)

Output Parameters — one per value to extract, **named exactly like the entry in the list**, each with the target where that value should go. For example:

Name	Target
`Yield`	`Batch.Yield`
`Lot`	`Batch.LotNumber`
`Operator`	`Batch.Operator`

Script

```
// Split a Value Property list:
//   [{"name":"Yield","value":"97"}, {"name":"Lot","value":"L-881"}, ...]
// into one output per entry. Name the step's Output Parameters exactly like the
// entries to extract; entries with no matching output are ignored.
const wanted = ['Yield', 'Lot', 'Operator']

// Value written when an entry is missing from the list.
const missing = ''

function parse(text) {
  if (text == null || String(text).trim() === '') return []
  const parsed = JSON.parse(text)
  if (!Array.isArray(parsed)) throw new Error('Expected a list of {name, value} pairs')
  return parsed
}

let entries = parse(list)

// Bound to a whole property such as ChildResults (rather than ChildResults.Value),
// the input arrives as [{"name":"Value","value":"[...]"}]: unwrap the inner list.
if (entries.length === 1 && entries[0].name === 'Value' &&
String(entries[0].value).trim().startsWith('[')) {
  entries = parse(entries[0].value)
}

const byName = {}
for (const entry of entries) {
  if (entry && typeof entry.name === 'string') byName[entry.name] = entry.value == null ? '' :
String(entry.value)
}

for (const name of wanted) {
  output[name] = Object.prototype.hasOwnProperty.call(byName, name) ? byName[name] : missing
}
```

With `ChildResults.Value` holding `[{"name":"Yield","value":"97"},{"name":"Lot","value":"L-881"}]`, the step writes `Batch.Yield = "97"`, `Batch.LotNumber = "L-881"` and `Batch.Operator = ""`.

To adapt it, list the entry names in `wanted` and give the step one Output Parameter per name. A list that is not valid JSON fails the step with the parse error, which a TRY rule can catch; change `missing` to a marker such as `'n/a'` if a blank value would be ambiguous.

---

7. Limits and troubleshooting

Limits

Limit	Value
Runtimes one runtime may create	100
Depth of a chain of runtimes creating runtimes	8
Starting parameters per create	200
Length of one starting-parameter value	10,000 characters

Hitting one of the first two limits returns code 3; too many or too long starting parameters return code 5. `Error.Text` says which limit was hit.

Common results

Code / text	Likely cause	Fix
The step fails with "Action server is not in the allowlist", or finds no action server	`http://mission-broker:3007` is not in the action-server allowlist, or the environment is not bound in this work center	Section 2.1, steps 1–3
2, work center not found	Misspelled name, or the work center is elsewhere in the tree	Use the full path `Plant/Area/Line`
2, workflow not found	The Master Workflow is not loaded in the target work center, or only Superseded/Obsolete versions are	Load it into the target work center
3, not permitted	No control grant to the target work center, or the grant is "only runtimes it created" and this runtime did not create the target	Add or widen the grant on the **Control** tab
4, wrong state	The command does not apply in the runtime's current state (section 3.1)	Read `Runtime.State` first, or retry Stop on an Unsuspending runtime
5, invalid input	A starting-parameter name the created workflow does not have, JSON that does not parse, or an ambiguous work-center name	Check the names against the created workflow's starting parameters
Outputs: 2, "outputs were not recorded"	The target finished before runtime outputs were recorded	Run it again